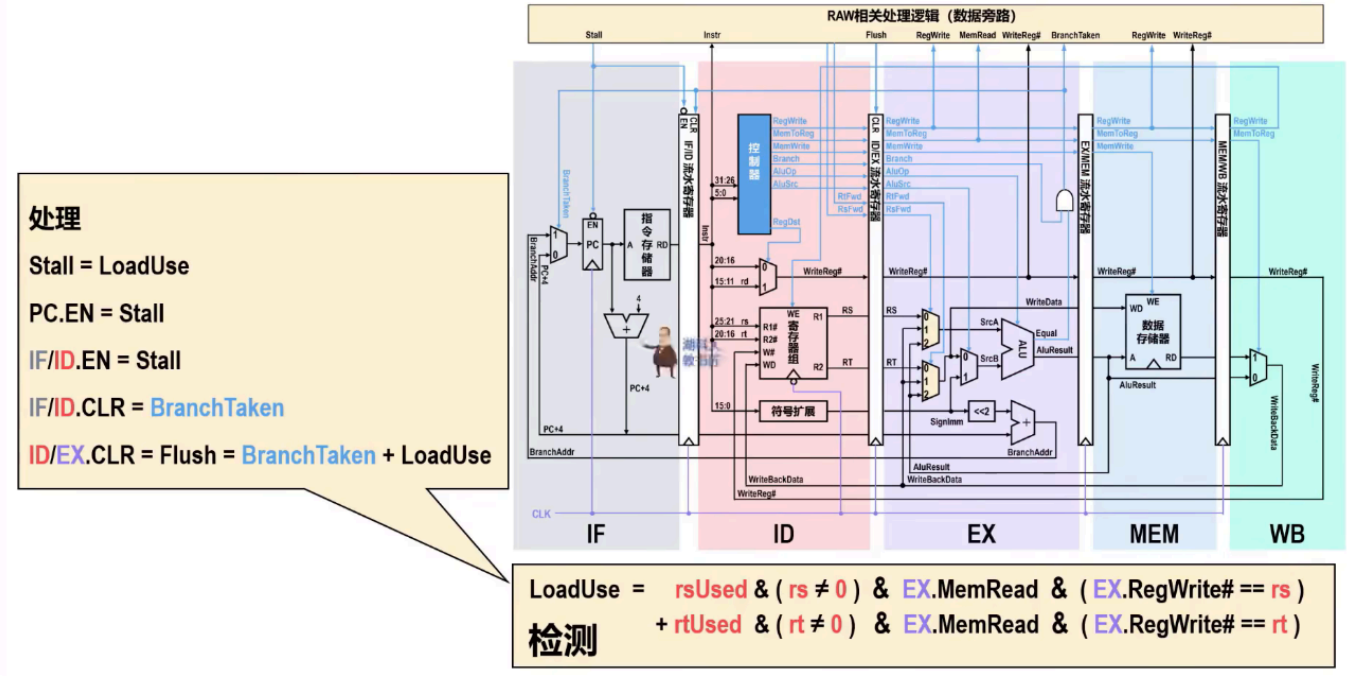
一、数据旁路法定义

RAW冲突的解决方案 —— 数据旁路

■ **数据旁路**是解决RAW冲突的另一种方法，也称为**数据转发**或**重定向**。

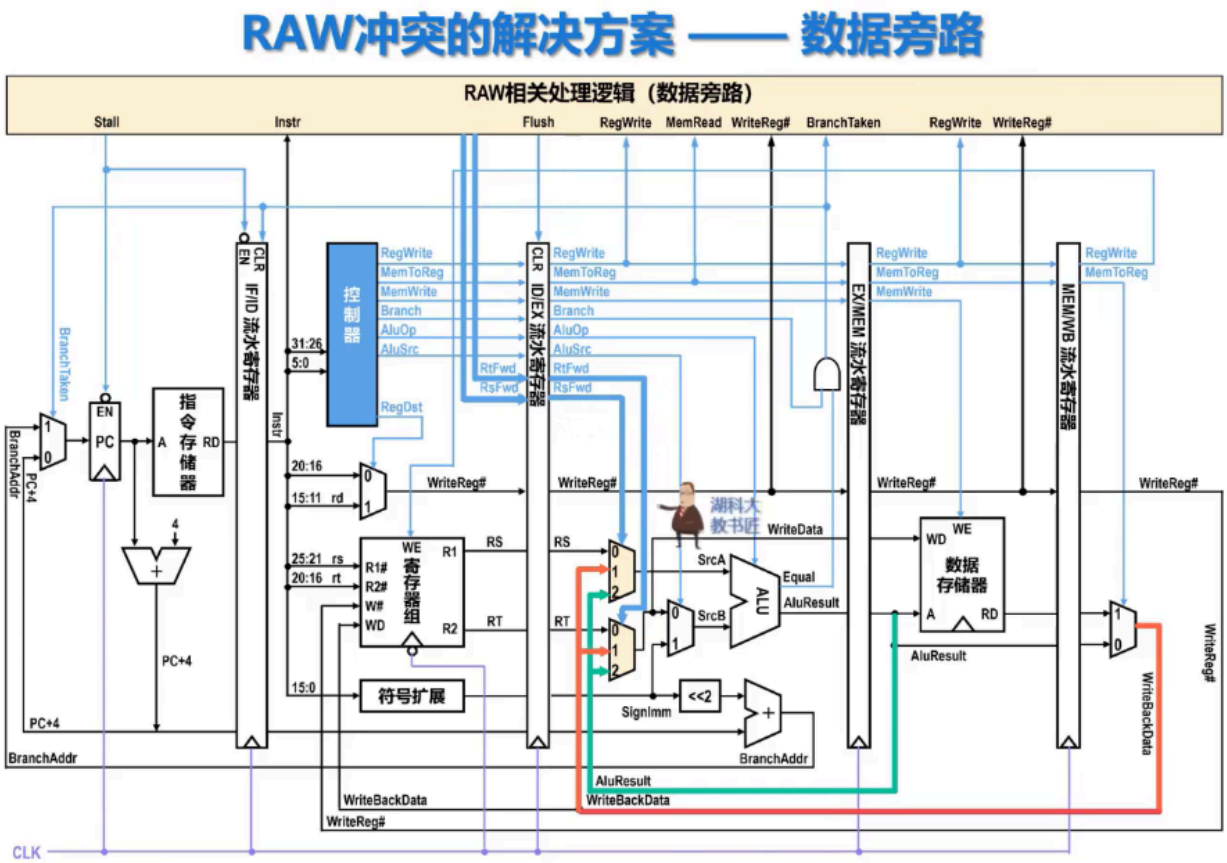
■ 与插入气泡的方法不同，**数据旁路的核心思想**是：与其让指令流水线“空转”（插入气泡）等待上一条指令将结果写回寄存器后再从寄存器读出，不如“**抄近道**”。既然上一条指令在EX功能段末尾就已经计算出了结果，那么就可以在内部将这个结果**直接**、**立刻**传送给下一条指令所需的输入端口。

二、增加了数据旁路的流水线框架



注：对于Load-Use型冲突，只能使用插入气泡法解决，所以加入了其检测方法

旁路的具体细节



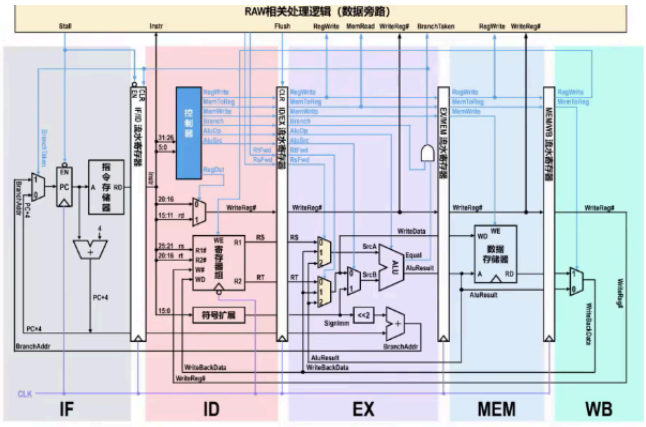
注：==本例的结构为基于执行指令（EX）阶段的，并未实现将分支指令提前到ID阶段的数据旁路==

旁路的处理

数据旁路处理

```
if ( rsUsed & ( rs ≠ 0 ) & EX.RegWrite & ( EX.RegWrite# == rs ) )
  RsFwd = 2
else if ( rsUsed & ( rs ≠ 0 ) & MEM.RegWrite & ( MEM.RegWrite# == rs ) )
  RsFwd = 1
else
  RsFwd = 0

if ( rtUsed & ( rt ≠ 0 ) & EX.RegWrite & ( EX.RegWrite# == rt ) )
  RtFwd = 2
else if ( rtUsed & ( rt ≠ 0 ) & MEM.RegWrite & ( MEM.RegWrite# == rt ) )
  RtFwd = 1
else
  RtFwd = 0
```



分支预测技术

RAW冲突的解决方案 —— 数据旁路

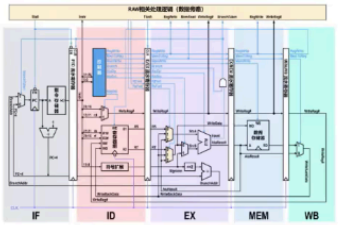
- 在采用**数据旁路**（也称为数据转发、重定向）机制优化指令流水线之后，RAW相关引起的数据冲突已基本解决，**仅在Load-Use相关时需要插入一个气泡**，**流水线效率因此显著提高**。此时，由分支指令引起的**控制冲突成为影响流水线性能的主要因素**。遵循“加速常见事件”的设计原则，应重点**减少分支指令带来的延迟损失**。为此，可考虑**将分支判断与地址计算等操作提前到ID功能段**完成。
- 为**进一步降低分支延迟**，通常采用**两种分支预测技术**：
  - ✓ **静态分支预测**：依赖于**编译器**提供的**静态信息**，对分支目标地址进行预测。此类**预测策略是固定不变的**，不随程序运行时的实际分支行为而调整，一般基于简单的规则或模式进行判断。
  - ✓ **动态分支预测**：通过追踪和分析分支指令的实际执行历史，在程序运行过程中**自适应地更新预测策略**。这种方法能够随着程序行为的演变而不断优化，因此通常**具备更高的预测准确率**。目前，动态分支预测已成为高性能处理器设计中广泛采用的关键技术。



三、举例

【练习】请给出该程序在右图所示数据旁路流水线中运行的时空图。

```
I1  lw $2, 8($1)        # M[R(1)+8] → R2
I2  add $3, $2, $4      # (R2)+(R4) → R3
I3  sub $5, $6, $7      # (R6)-(R7) → R5
I4  or  $8, $3, $9      # (R3)|(R9) → R8
I5  and $10, $8, $11    # (R8)&(R11) → R10
```



提示：

对于**Load-Use相关**，**不能采用数据旁路**方式进行消除，**仍需采用插入气泡方式**进行消除。

时钟周期	IF段	ID段	EX段	MEM段	WB段
1	lw $2, 8($1)				
2	add $3, $2, $4	lw $2, 8($1)			
3	sub $5, $6, $7	add $3, $2, $4	lw $2, 8($1)		
4	sub $5, $6, $7	add $3, $2, $4	nop	lw $2, 8($1)	
5	or $8, $3, $9	sub $5, $6, $7	add $3, $2, $4	nop	lw $2, 8($1)
6	and $10, $8, $11	or $8, $3, $9	sub $5, $6, $7	add $3, $2, $4	nop
7	下一条指令	and $10, $8, $11	or $8, $3, $9	sub $5, $6, $7	add $3, $2, $4



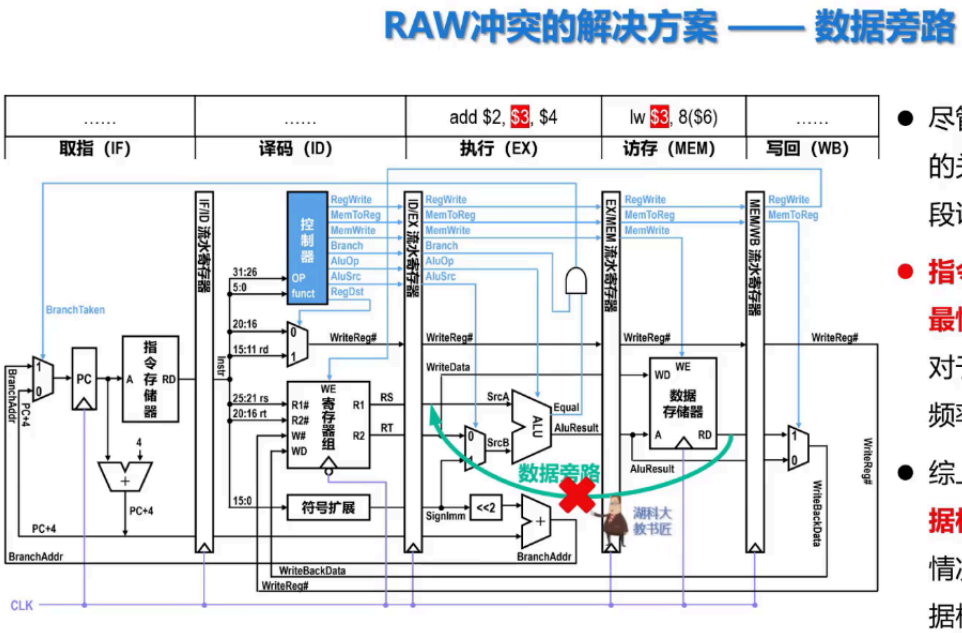
四、不同数据相关的处理方式

数据相关	指令序列举例	指令距离	气泡法（无旁路）插入气泡数量	数据旁路法插入气泡数量	旁路路径/备注
相邻R-R RAW	and $3, $1, $2 sub $4, $3, $0	0	2 (EX段和MEM段各1个)	0	EX/MEM流水寄存器 → ALU输入端
非相邻R-R RAW	and $3, $1, $2 1条无数据相关的指令 add $5, $3, $3	1	1 (EX段)	0	MEM/WB流水寄存器 → ALU输入端
远距离R-R RAW	and $3, $1, $2 至少2条无数据相关的指令 or $6, $7, $3	≥2	0	0	无需数据旁路，利用CPU时钟周期的功能划分（如下跳沿时写入、低电平时读出）解决ID段与WB段的RAW冲突
相邻Load-Use	lw $3, 8($6) add $2, $3, $4	0	2 (EX段和MEM段各1个)	1 (EX段)	MEM/WB流水寄存器 → ALU输入端
非相邻Load-Use	lw $3, 8($6) 1条无数据相关的指令 add $2, $3, $4	1	1 (EX段)	0	MEM/WB流水寄存器 → ALU输入端
远距离Load-Use	lw $3, 8($6) 至少2条无数据相关的指令 add $2, $3, $4	≥2	0	0	无需数据旁路，利用CPU时钟周期的功能划分（如下跳沿时写入、低电平时读出）解决ID段与WB段的RAW冲突

以上不包含将分支指令（如beq、bne）提前到ID段执行的情况。



关于load-use必须加入气泡



- 尽管可以实现该功能，但**EX功能段**的关键路径**延迟增大**到了MEM功能段访存延迟加EX功能段的ALU延迟。
- **指令流水线的频率取决于流水线中最慢的功能段**（短板效应）。因此，对于图中所示的情况，会使流水线频率严重降低。
- 综上所述，将相邻两条指令存在**数据相关**且**前一条指令是访存指令**的情况，称为**Load-Use相关**，这种数据相关**不能采用数据旁路处理**。

为什么封死？——关键路径会变长

> **"尽管可以实现该功能，但 EX 功能段的关键路径延迟增大到了 MEM 功能段访存延迟加 EX 功能段的 ALU 延迟"**

这句话的含义是：

如果要在同一拍内完成"MEM 读出数据 → 旁路 → ALU 使用"，那么 **ALU 的输入就必须等访存的结果出来后才能开始运算**

这就导致 EX 段的**关键路径**（决定时钟周期长度的那条最长信号路径）从原来单纯的：

```
寄存器读出 → ALU 运算 → 输出结果
```

变成了：

```
寄存器读出 → ALU 运算(部分) → 等MEM访存完成 → 旁路数据送达 → ALU 运算(剩余)
```

也就是 **EX 段延迟 = ALU 延迟 + 访存延迟（两者串联）**

为什么关键路径变长是致命问题?

右侧第二条要点：

> **"指令流水线的频率取决于流水线中最慢的功能段（短板效应）。因此，对于图中所示的情况，会使流水线频率严重降低"**

流水线的时钟周期由**最慢的一段**决定。原本 5 段中每段延迟大致均衡：

```
IF ≈ ID ≈ EX ≈ MEM ≈ WB  →  时钟周期 = 任意一段的延迟
```

如果 EX 段吸收了访存延迟，变成：

```
EX' = EX + MEM  →  EX' 变成最慢的段
时钟周期 = EX' = EX + MEM  →  几乎是原来的两倍
```

频率直接**腰斩**，所有指令都跟着慢了一倍

和之前"时间差"解释的关系

两个角度说的是同一件事的**两面**：

角度	说法
时间角度（之前讲的）	ALU 在第4拍中段需要数据，但访存到第4拍末才出数据 → 赶不上
硬件延迟角度（这张图讲的）	如果非要让 ALU 等访存结果再运算，那 EX 段的关键路径 = ALU延迟 + 访存延迟 → 时钟周期被迫拉长

前者解释"为什么赶不上"，后者解释"即使你硬要赶，代价是什么"

一句话总结

不是物理上做不到这条旁路线，而是如果允许数据从 MEM 段直接旁路回 EX 段的 ALU，就会把"访存延迟 + ALU 延迟"叠加到同一条关键路径上，使 EX 段成为整个流水线的瓶颈，**时钟频率严重降低。**所以工程上的选择是宁愿插入一个气泡（stall 一拍），也不愿意让全局降频